

**SIMATS ENGINEERING**  
**Department of Computer Science and Engineering**  
**ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING**

|                         |                                                                                                                                                                                                                             |                      |                            |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|----------------------------|
| <b>Course Code:</b>     | CSA1220                                                                                                                                                                                                                     | <b>Course Title:</b> | Computer Architecture      |
| <b>CO Assessed:</b>     | CO2 – Precision (BL3)                                                                                                                                                                                                       | <b>Assessment:</b>   | Analytical Problem Solving |
| <b>Weightage:</b>       | 45%                                                                                                                                                                                                                         | <b>Total Marks:</b>  | 100                        |
| <b>CO2 Statement:</b>   | Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. |                      |                            |
| <b>Student Name:</b>    | LOKESH S                                                                                                                                                                                                                    | <b>Reg. No.:</b>     | 192572151                  |
| <b>Section / Batch:</b> | CSE(AI) / 2 <sup>ND</sup> YEAR                                                                                                                                                                                              | <b>Date:</b>         | 03/08/2026                 |

**Instructions to Students**

- Identify the appropriate arithmetic algorithm or number representation required for the given computational problem.
- Analyse the step-by-step execution of integer and floating-point arithmetic operations using the selected algorithm.
- Apply Booth's multiplication, restoring/non-restoring division, and IEEE 754 standards to obtain accurate results.
- Compute binary multiplication, division, normalization, and floating-point arithmetic using appropriate algorithms.
- Interpret the computed results by evaluating their accuracy, precision, and suitability for the given application.

**QUESTIONS**

1. A digital computer performs signed integer arithmetic using 8-bit 2's complement representation. Consider the numbers -75 and +38. Analyze how the processor performs both addition and subtraction by converting the operands to binary, executing the operations step-by-step, detecting overflow (if any), and interpreting the final results in both binary and decimal forms.
2. A high-performance processor employs an 8-bit carry look-ahead adder (CLA) to enhance arithmetic speed. Given two binary operands, analyse how the generate and propagate signals are produced, how carry bits are calculated simultaneously, and compare the execution time and hardware complexity of both adders. Explain why CLA is widely used in modern processors.
3. A digital processing unit executes signed multiplication using Booth's algorithm. Multiply +13 and -6, and analyze the algorithm step-by-step by showing the contents of the accumulator, multiplier, multiplicand, and Booth bit after each iteration. Explain the purpose of arithmetic right shifts and discuss how Booth's algorithm reduces the number of addition and subtraction operations.
4. A binary arithmetic unit supports both restoring and non-restoring division methods for signed integer operations. Analyze the division of the binary numbers  $27 \div 5$  using each algorithm by showing the intermediate remainder values, quotient formation, and register contents after every iteration. Compare the two methods based on execution steps, computational complexity, and hardware efficiency, and justify which algorithm is more suitable for high-speed processors.

5. A floating-point processing unit represents real numbers using the IEEE 754 standard. Analyze the representation of the decimal number  $-42.375$  in both single-precision (32-bit) and double precision (64-bit) formats by identifying the sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between  $-42.375$  and  $+15.125$ , including exponent alignment, mantissa operations, normalization, rounding, and the impact of precision on the final result.

### Code of Conduct

*I **LOKESH S (192572151)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the Student: LOKESH S**

### RUBRICS FOR CALCULATION

| Criteria                  | Weightage | Level 4 – Exemplary (5 Marks)                                                     | Level 3 – Proficient (4 Marks)            | Level 2 – Developing (2–3 Marks)           | Level 1 – Beginning (0–1 Mark)                |
|---------------------------|-----------|-----------------------------------------------------------------------------------|-------------------------------------------|--------------------------------------------|-----------------------------------------------|
| Problem Understanding     | 20%       | Clearly interprets problem, identifies all parameters and requirements accurately | Understands problem with minor gaps       | Partial understanding; misses key elements | Misinterprets or unable to understand problem |
| Method Selection          | 20%       | Selects most appropriate method/formula with strong justification                 | Selects correct method with minor errors  | Inappropriate or partially correct method  | Unable to select method                       |
| Solution Process          | 25%       | Logical, systematic steps with correct approach throughout                        | Mostly correct steps with minor mistakes  | Disorganized steps; multiple errors        | No clear process                              |
| Accuracy of Calculation   | 20%       | All calculations accurate; correct final answer                                   | Minor calculation errors                  | Frequent errors; incorrect final answer    | No valid calculations                         |
| Interpretation of Results | 15%       | Clearly interprets results with units and engineering relevance                   | Basic interpretation with minor omissions | Limited or unclear interpretation          | No interpretation                             |

### Marks Evaluation:

| <b>Q. No</b> | <b>Problem Understanding (20)</b> | <b>Method Selection (20)</b> | <b>Solution Process (25)</b> | <b>Accuracy of Calculation (20)</b> | <b>Interpretation of Results (15)</b> | <b>Total (out of 100)</b> |
|--------------|-----------------------------------|------------------------------|------------------------------|-------------------------------------|---------------------------------------|---------------------------|
| <b>1</b>     | / 4                               | / 4                          | / 4                          | / 4                                 | / 4                                   |                           |
| <b>2</b>     | / 4                               | / 4                          | / 4                          | / 4                                 | / 4                                   |                           |
| <b>3</b>     | / 4                               | / 4                          | / 4                          | / 4                                 | / 4                                   |                           |
| <b>4</b>     | / 4                               | / 4                          | / 4                          | / 4                                 | / 4                                   |                           |
| <b>5</b>     | / 4                               | / 4                          | / 4                          | / 4                                 | / 4                                   |                           |
|              |                                   |                              |                              |                                     |                                       |                           |
|              | / 20                              | / 20                         | / 25                         | / 20                                | / 15                                  | / 100                     |

**Course Coordinator Faculty Incharge**

# ① 8-bit 2's complement Arithmetic

Given,

$$-75 = 10110101$$

$$+38 = 00100110$$

## Addition

$$\begin{array}{r} -75 \rightarrow 10110101 \\ 38 \rightarrow 00100110 \\ \hline -37 \quad 11011011 \end{array}$$

The result is negative, no overflow.

$$\begin{array}{r} -37 \Rightarrow 11011011 \\ 1's \quad 00100100 \\ 2's \quad 1 \\ \hline 11011011 \\ 00100101 \\ \hline \end{array}$$

## Subtraction

$$-75 + 38 \Rightarrow -75 + (-38)$$

2's complement of 38.

$$-38 = 11011010$$

$$\begin{array}{r} 10110101 \\ 11011010 \\ \hline \text{Carry } \textcircled{1} 10001111 \end{array}$$

There is no overflow

## **1. SIGNED ARITHMETIC USING 8-BIT 2'S COMPLEMENT**

- 2's complement is the most common method for representing signed integers in computers.
- Positive numbers are stored in normal binary form, while negative numbers are obtained by taking the 1's complement and adding 1.
- Using a single binary adder, the processor performs both addition and subtraction, simplifying hardware design.
- Subtraction is carried out by adding the 2's complement of the second operand instead of performing direct subtraction.
- Overflow occurs only when two numbers having the same sign produce a result with the opposite sign.
- The valid range for 8-bit signed numbers is -128 to +127.
- This representation eliminates the need for separate subtraction circuits and supports efficient arithmetic.
- Modern processors and ALUs use 2's complement because it is fast, accurate and easy to implement.
- Applications include processors, embedded systems, controllers and digital computers.
- Key points: binary conversion, addition, subtraction, overflow detection and result interpretation.

## **CONCLUSION**

This topic is important in computer arithmetic because it improves the speed, accuracy and efficiency of arithmetic operations performed by modern processors.

$$\Rightarrow -113 = 10001111.$$

2) 8-bit Carry Look Ahead Adder

$G_i = A_i \cdot B_i \rightarrow$  Generate a carry regardless of  $C_i$

$P_i = A_i \oplus B_i \rightarrow$  propagate an incoming carry

$$A = 00100110$$

$$B = 10110101$$

| Bit | i | $A_i$ | $B_i$ | $G_i$ | $P_i$ |
|-----|---|-------|-------|-------|-------|
| 0   |   | 0     | 1     | 0     | 1     |
| 1   |   | 1     | 0     | 0     | 1     |
| 2   |   | 1     | 1     | 1     | 0     |
| 3   |   | 0     | 0     | 0     | 0     |
| 4   |   | 0     | 1     | 0     | 1     |
| 5   |   | 1     | 1     | 1     | 0     |
| 6   |   | 0     | 0     | 0     | 0     |
| 7   |   | 0     | 1     | 0     | 1     |

Carry Equation

$$\rightarrow C_1 = G_{10} + P_0 C_0 = 0$$

$$C_2 = G_{11} + P_1 C_1 = 0$$

$$C_3 = G_{12} + P_2 C_2 = 1$$

$$C_4 = G_{13} + P_3 C_3 = 0$$

$$C_5 = G_{14} + P_4 C_4 = 0$$

$$C_6 = G_{15} + P_5 C_5 = 1$$

$$C_7 = G_{16} + P_6 C_6 = 0$$

$$C_8 = G_{17} + P_7 C_7 = 0$$

## 2. CARRY LOOK-AHEAD ADDER (CLA)

- A Carry Look-Ahead Adder is designed to reduce carry propagation delay during binary addition.
- Each bit generates Generate (G) and Propagate (P) signals.
- Generate indicates a carry is produced, while Propagate forwards an incoming carry.
- All carry bits are calculated simultaneously using combinational logic.
- This greatly reduces execution time compared with Ripple Carry Adders.
- CLA provides high-speed arithmetic operations in CPUs and ALUs.
- The design requires more logic gates and hardware than Ripple Carry Adders.
- Hardware complexity is higher but performance is significantly improved.
- Modern processors use CLA to increase arithmetic speed.
- Key points: Generate, Propagate, simultaneous carry calculation, speed, hardware complexity and applications.

## CONCLUSION

This topic is important in computer arithmetic because it improves the speed, accuracy and efficiency of arithmetic operations performed by modern processors.

3) Multiplicand  $M = +13 \Rightarrow 01101 \Rightarrow -M = 10011$

Multiplicand (6)  $Q = 11010$

$A = 00000$

$Q_0 = 0$

| n | A     | Q     | $Q_0$ | Action         |
|---|-------|-------|-------|----------------|
|   |       |       |       | Initialization |
| 5 | 00000 | 11010 | 0     | $A = A - M$    |
|   | 11001 | 10110 | 0     |                |
|   | 11100 | 11011 | 0     | ASR            |
| 4 | 00011 | 01011 | 0     | $A = A + M$    |
|   | 11001 | 01101 | 0     | $A = A + M$    |
| 3 | 11001 | 01101 | 0     |                |
|   | 11011 | 00101 | 1     | $A = A - M$    |
| 2 | 11011 | 00101 | 1     |                |
|   | 11101 | 10010 | 1     | ASR            |
| 1 | 11101 | 10010 | 1     |                |

Final product  $(AQ) \Rightarrow 1110110010 \Rightarrow -78$

4) Given

Dividend  $Q = 27 = 11011$

Divisor  $M = 5 = 00101$

$A = 00000$

### **3. BOOTH'S ALGORITHM**

- Booth's algorithm performs signed multiplication using 2's complement representation.
- It uses four registers: Accumulator (A), Multiplier (Q), Multiplicand (M) and Booth bit (Q-1).
- The pair Q0 and Q-1 determines whether addition, subtraction or no operation is performed.
- Rules: 00 and 11 -> No operation, 01 -> Add multiplicand, 10 -> Subtract multiplicand.
- After every operation an arithmetic right shift is performed.
- The arithmetic shift preserves the sign bit of the partial product.
- The algorithm reduces repeated additions when consecutive 1s appear in the multiplier.
- This improves multiplication efficiency and execution speed.
- It is widely used in processors and digital signal processors.
- Key points: signed multiplication, arithmetic shift, Booth bit and reduced operations.

### **CONCLUSION**

This topic is important in computer arithmetic because it improves the speed, accuracy and efficiency of arithmetic operations performed by modern processors.

| A     | Q     | Action                            |
|-------|-------|-----------------------------------|
| 00000 | 11011 | Initial                           |
| 00001 | 10110 | Negative, Restore A, $Q_0 = 0$    |
| 00011 | 01100 | Negative, Restore A, $Q_0 = 0$    |
| 00001 | 11001 | Positive, $Q_0 = 1$ , $A = A - M$ |
| 00011 | 10010 | Negative, Restore A, $Q_0 = 0$    |
| 00010 | 00101 | Positive, $Q_0 = 1$               |

Quotient = 00101  $\Rightarrow 5$

Remainder = 00010  $\Rightarrow 2$

5) Decimal to Binary.

$$42.375 = (101010.011)_2$$

Normalized form

$$1.01010011 \times 2^5$$

single precision (32-bit)

→ Sign bit = 1

→ Exponent =  $5 + 127 = 132$

→ Exponent binary = 10000100

→ Mantissa = 010100110000000000000000

32 bit representation.

#### **4. RESTORING AND NON-RESTORING DIVISION**

- Binary division can be implemented using restoring and non-restoring algorithms.
- Restoring division restores the remainder whenever subtraction gives a negative value.
- Non-restoring division avoids immediate restoration and corrects only when required.
- Both methods repeatedly shift registers and update quotient bits.
- Restoring division is simpler but requires more arithmetic operations.
- Non-restoring division executes faster because unnecessary restoration is avoided.
- Hardware utilization is better in non-restoring division.
- Both algorithms produce the same quotient and remainder.
- High-speed processors generally prefer non-restoring division.
- Key points: remainder update, quotient formation, execution speed and hardware efficiency.

#### **CONCLUSION**

This topic is important in computer arithmetic because it improves the speed, accuracy and efficiency of arithmetic operations performed by modern processors.

Now, Double precision (64-bit)

→ sign bit = 1

→ Exponent =  $5 + 1023 = 1028$

→ Exponent Binary = 10000000100.

→ Mantissa = 01010011000000000000000000000000

64 bits representation.

## **5. IEEE 754 FLOATING-POINT REPRESENTATION**

- IEEE 754 is the standard format for representing real numbers in computers.
- Single precision uses 32 bits: 1 sign bit, 8 exponent bits and 23 mantissa bits.
- Double precision uses 64 bits: 1 sign bit, 11 exponent bits and 52 mantissa bits.
- The sign bit identifies positive or negative numbers.
- The exponent stores the biased exponent value.
- The mantissa stores the significant digits of the number.
- Floating-point addition involves exponent alignment, mantissa addition or subtraction, normalization and rounding.
- Double precision provides greater accuracy and lower rounding error than single precision.
- IEEE 754 is widely used in scientific, engineering and graphics applications.
- Key points: sign, exponent, mantissa, normalization, rounding and precision.

## **CONCLUSION**

This topic is important in computer arithmetic because it improves the speed, accuracy and efficiency of arithmetic operations performed by modern processors.